

Protobufs Explained



Giga Gatenashvili · Follow

Published in Level Up Coding

8 min read · Nov 14, 2022



Listen



Share



More

In essence Protocol Buffers, aka protobufs, allow encoding of structured data into a compact byte stream (a sequence of zeroes and ones). Say, you have a player JavaScript object with properties score and name:

```
{  
  score : 25,  
  name: "Tom"  
}
```

If serialized through protobufs it would turn into a byte stream like the following (each byte represented with two hex digits):

```
08 C8 01 12 03 54 6F 6D
```

As you can see it's 8 bytes altogether.

For comparison the same object serialized in JSON (whitespace removed) would result in 26 bytes of data. So in this particular example protobufs variant is roughly x3 smaller. This difference will vary depending on underline data, but overall protobuf serialized version is always smaller than its text-based counterparts and will make huge difference on large-scale systems.

Some of the questions you might have about protobufs even if you have used them for some time now:

- Why do we need proto definitions, could not we convert structured messages directly into binary?
- Why do we need field numbers in proto definitions?
- Why do we need to define types for fields in addition to language native types?
- Distinction between native types vs proto types vs wire types.
- ...

Encoding

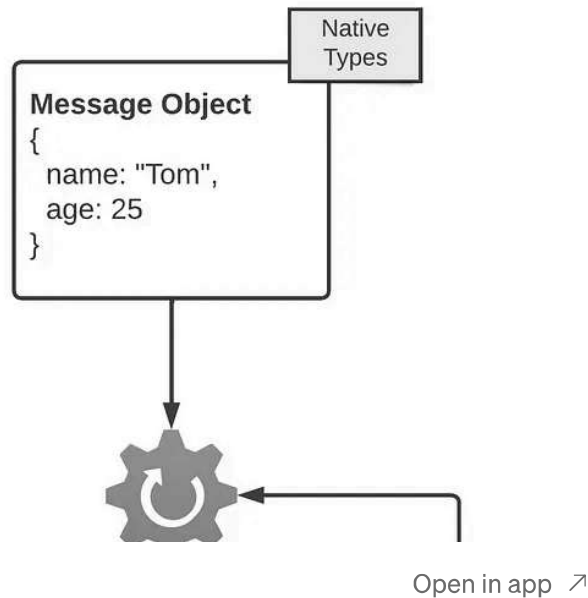
To encode the above object into a byte stream, a protobuf encoder requires two inputs, the actual object to be encoded and its proto definition.

Proto definitions are written using proto language. There are two versions of the language proto2 and proto3 we will be talking the latter (though the examples we use here will be the same in proto2)

Here is what proto definition for the above object looks like

```
message Player {  
    int32 score = 1;  
    string name = 2;  
}
```

And here is a high level view on encoding



Medium

Search



```
int32 age = 2;
}
```

Let's talk a bit about proto language and its types, following our example, structured data is defined as a `message`, individual fields have types and field numbers, “name” is of type `string` and its field number is 1; “age” is of type `int32` with field number 2. These types are language-neutral, meaning, a `message` type can be translated into an object in JavaScript and POJO or class in Java, `int32` can be translated into a `number` in JavaScript, while in Java it can have a more specific `int` type.

3 typing systems

We need to keep in mind that there are multiple typing systems involved at encoding/decoding.

- **wire-types** — how things are written in a byte stream
- **proto types** — proto definition language-neutral types, kinda instructions to protobufs encoder/decoder on how to turn language native types into wire-types and back.
- **language native types** — e.g. JavaScript object, string, number, boolean or Java class, String, char, int, float, boolean

One interesting point to think about is, **wire-types** and **language native types** have memory representations (e.i. how they are stored in 0s and 1s), while **proto types** never exist in memory, they are used during encoding to turn language native types into wire-types and during decoding to do the opposite.

We will see what roles all three typing systems play during encoding/decoding shortly when we go over our example, but before that we need to understand some basics. We will start with varints.

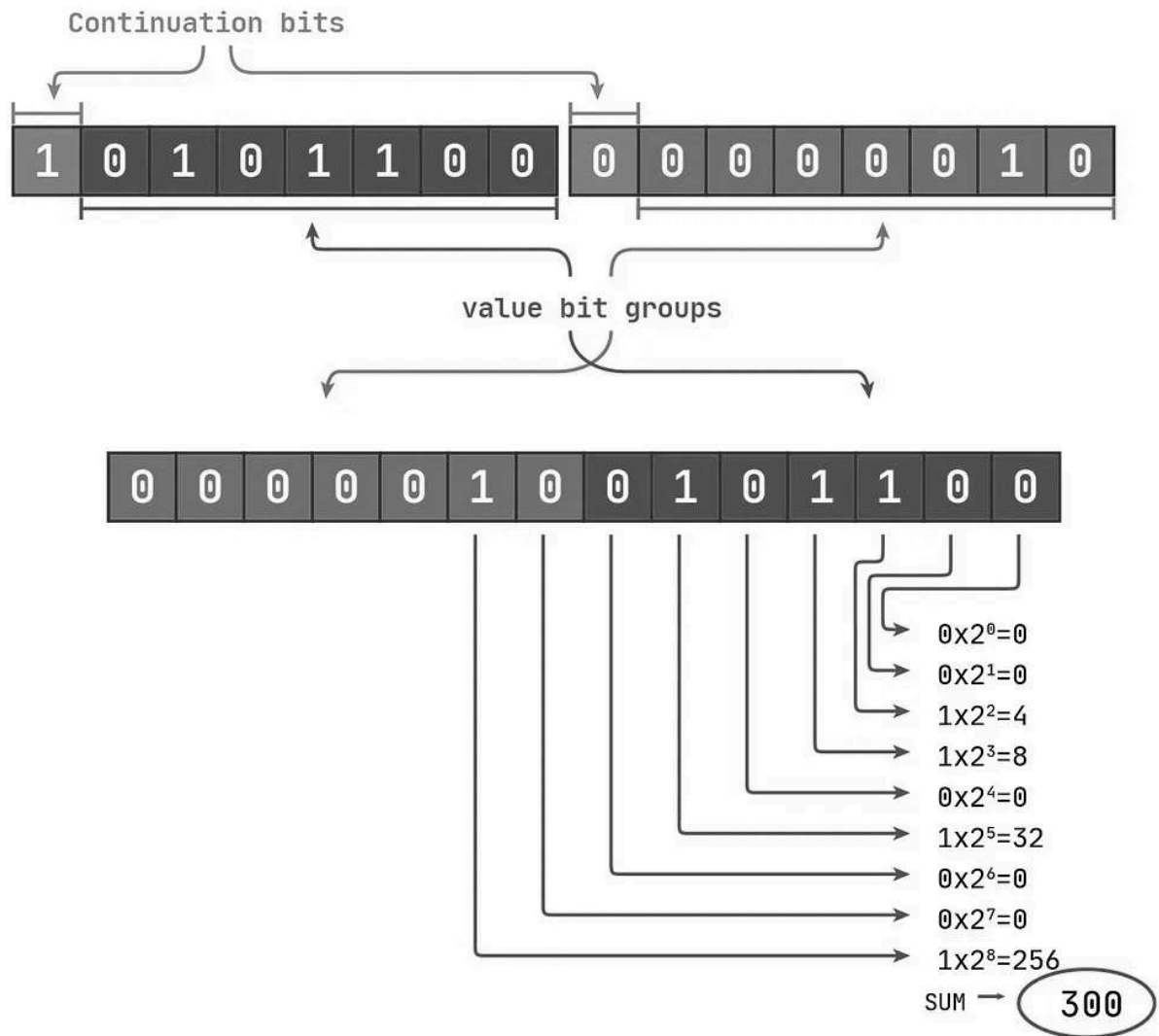
Varints

Varints are a method of serializing integers using one or more bytes. A varint's byte length increases as the integer gets bigger. As such, varint might consist with one or more bytes, the most significant bit (MSB) in each byte is called continuation bit, when it's set it means there is at least one more bytes to read, rest 7 bits in each byte are dedicated for storing the value. Highest unsigned value you can store in 7 bits is $2^7=128$ so anything higher than that will require another byte to fit in. Let's go over examples, to see how this actually works.

The integer number `1` encoded in a varint is `00000001` to figure it out we look at the most significant bit and see that it's zero, which tells us that this varint has no more bytes for reading, next we drop the most significant bit (as it's used only for properly reading the varint) and we are left with 7 bits of actual integer value `0000001` which is `1` in base10.

Now let's look at how we can figure out that varint `1010 1100 0000 0010` actually holds integer value `300`. We look at the first bit in the first byte and see that it's set, which means that there is another byte to read, then we read the next byte at its first bit and see that it's not set, telling us that we have reached the end. Next we drop continuation bits and what's left is groups of bits that hold the integer value, we reverse them because varints store less significant groups first, and that's our final integer in base2, then we can use positional notation method to see what it looks like in base10. The following image illustrates that.

300 encoded in varint wire-type



understanding varint

Need more about varints? Look at [Carl Mastrangelo](#) write up, it covers many interesting aspects.

Encoded message basics

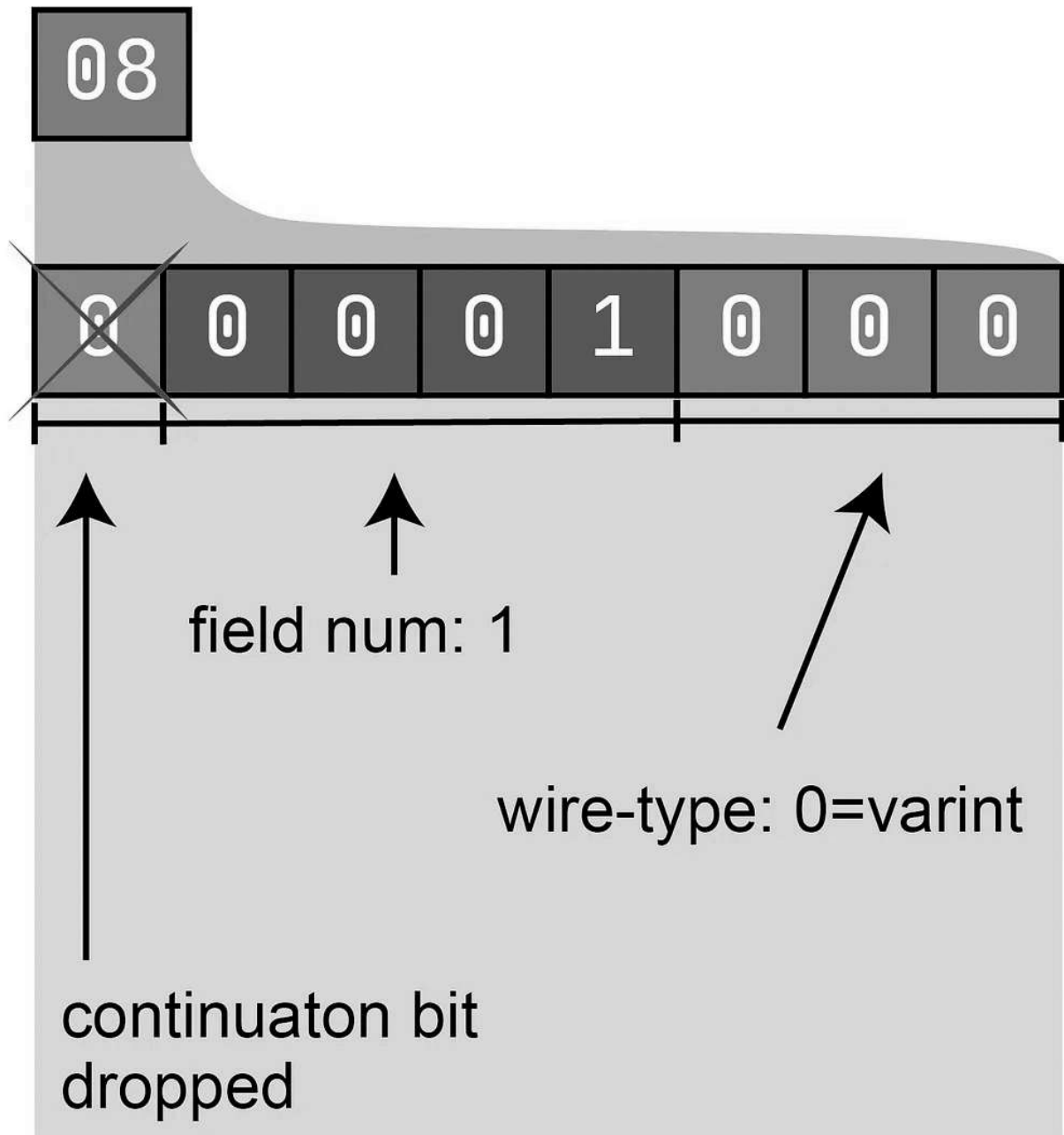
When a message gets encoded each field is written as a key-value pair, one after another. Each key in the byte stream is a varint with value $(\text{field_number} \ll 3) | \text{wire_type}$, that is, last 3 bits always store wire-type of the value. **Order of key-value pairs is not guaranteed.**

Reading the first key-value pair

I propose we pretend to be a protobufs decoder for a bit and read our example byte stream just like it would. In addition to that let's imagine that we have no access to

the proto definitions of the message for now, only things we know is what the byte stream looks like and how protobufs protocol works.

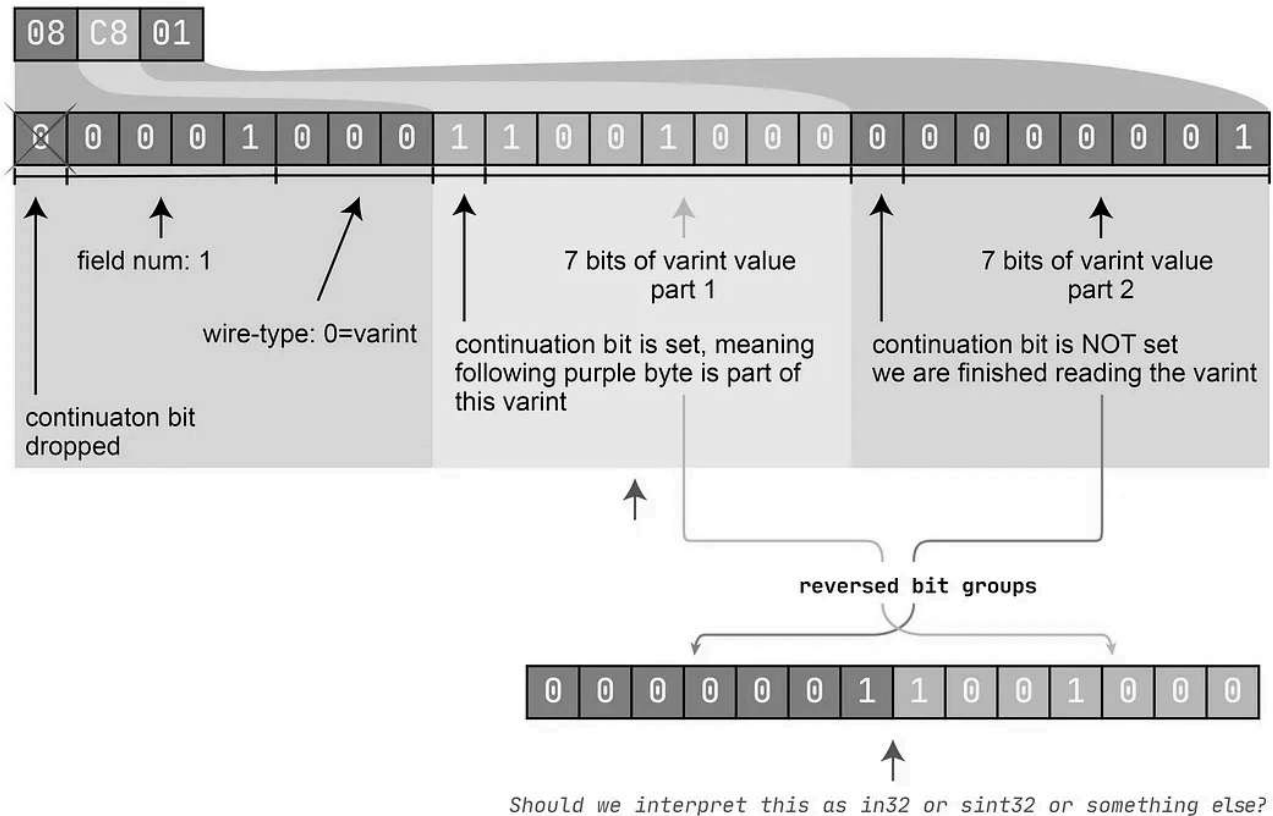
In our example the first byte is `08`, from what we already said we expect it to be a field key along with the `wire_type`.



The field-key

At this point we know that field number is 1 and wire-type is 0 which corresponds to Varint (as defined by the protobufs protocol), based on this we can go ahead and read varint that comes next in the byte stream `c8` which is `11001000` its continuation

bit is set — we need to read next byte too 01 which is 00000001 this time
 continuation bit is not set — we are done reading the varint. Next we need to drop continuation bits and reverse order of bit groups to get 0000001 1001000 (to understand why I'm not converting this to base10 yet, just read on)



What name corresponds to field num: 1?

We can't know these unless we look at the proto definition.

With this we are done with reading first encoded key-value pair, but that's just reading, the original object property reconstruction still requires a few more steps.

Protobuf messages are not fully self describing

A decoder can not fully reconstruct original message without looking at its proto definitions. Let's look what information we've gathered so far. We've read anything related to this field from the byte stream, but is this enough information to reconstruct the original object property?

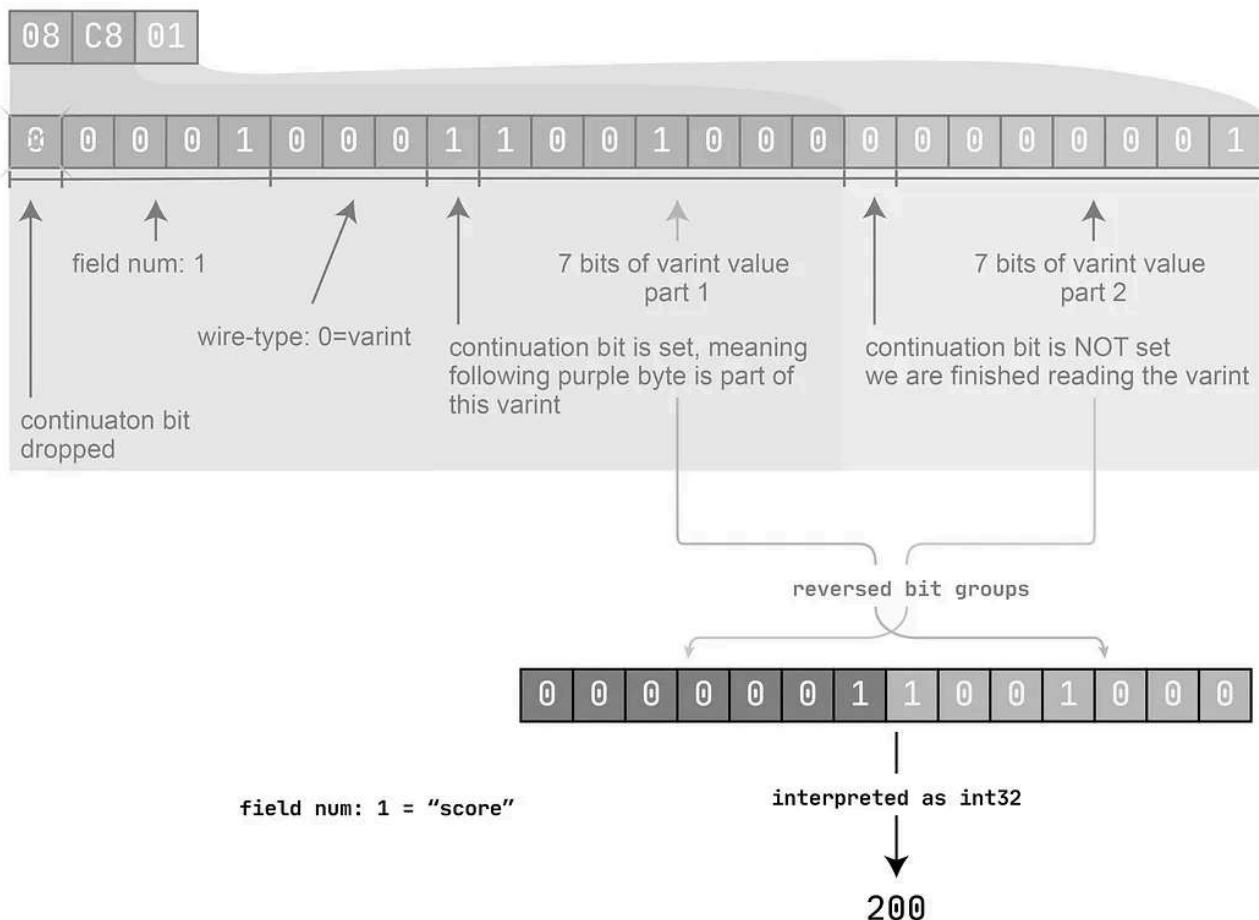
- Field number: 1
- Wire-type: Varint

- Value: 0000001 1001000

Right, we still need to figure out field name and how to interpret the binary value.

To make it clear why we can't interpret the value without looking at the field's proto type definition, consider this, proto types `int32` and `sint32` both get encoded in varint wire-type. Now the value 0000001 1001000 can mean 200 for `int32` and 100 for `sint32`. You would not be happy if a 100 bucks disappeared from your bank account just because someone had no access to proto definitions. Remember we said that proto types are kind of instructions for encoder/decoder, there you go.

At this point doors are open to look at the proto definitions, from it we can tell that field with number 1 is named "score", and its proto type is `int32`.



Say we're decoding in JavaScript environment, the type tells us that we should interpret the value as a normal varint (if it was `sint32` we would need one additional step to decode it as ZigZag signed integer encoding which would result in value 100)

it would also instruct us to use JS `number` type as the result type. So we end up assigning property score with number value 200 to the target object.

For awareness this is what types look like for this particular field

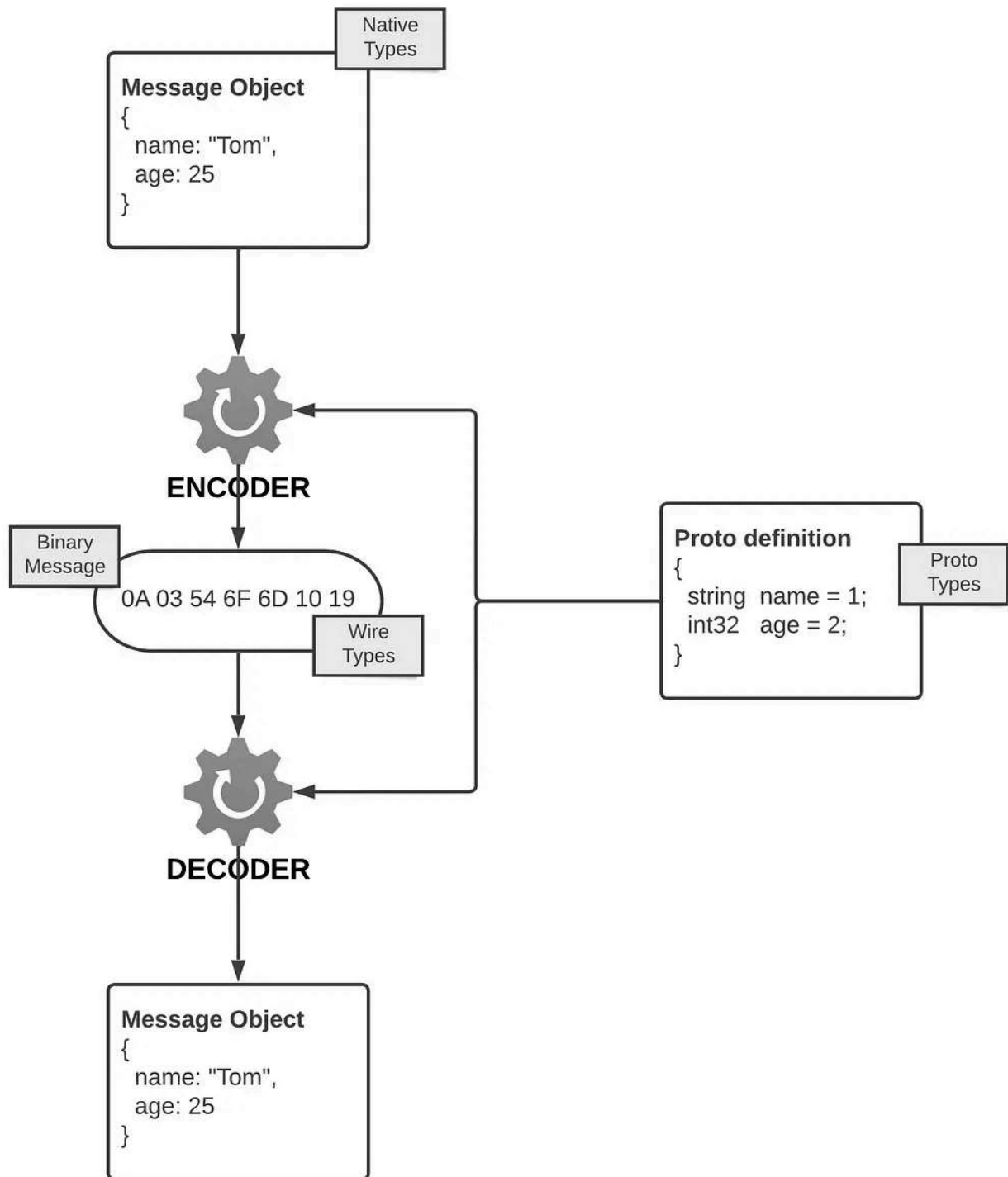
JavaScript	proto type	wire-type
number	int32	Varint

Proto definitions are used by the encoder to:

- Validate input, e.g. JavaScript value of type `number` fits proto `int32` but `string` does not.
- Determine target wire-type, e.g. `Varint` for `int32`
- Include field numbers, to make field identification possible by the decoder

They are used by the decoder to:

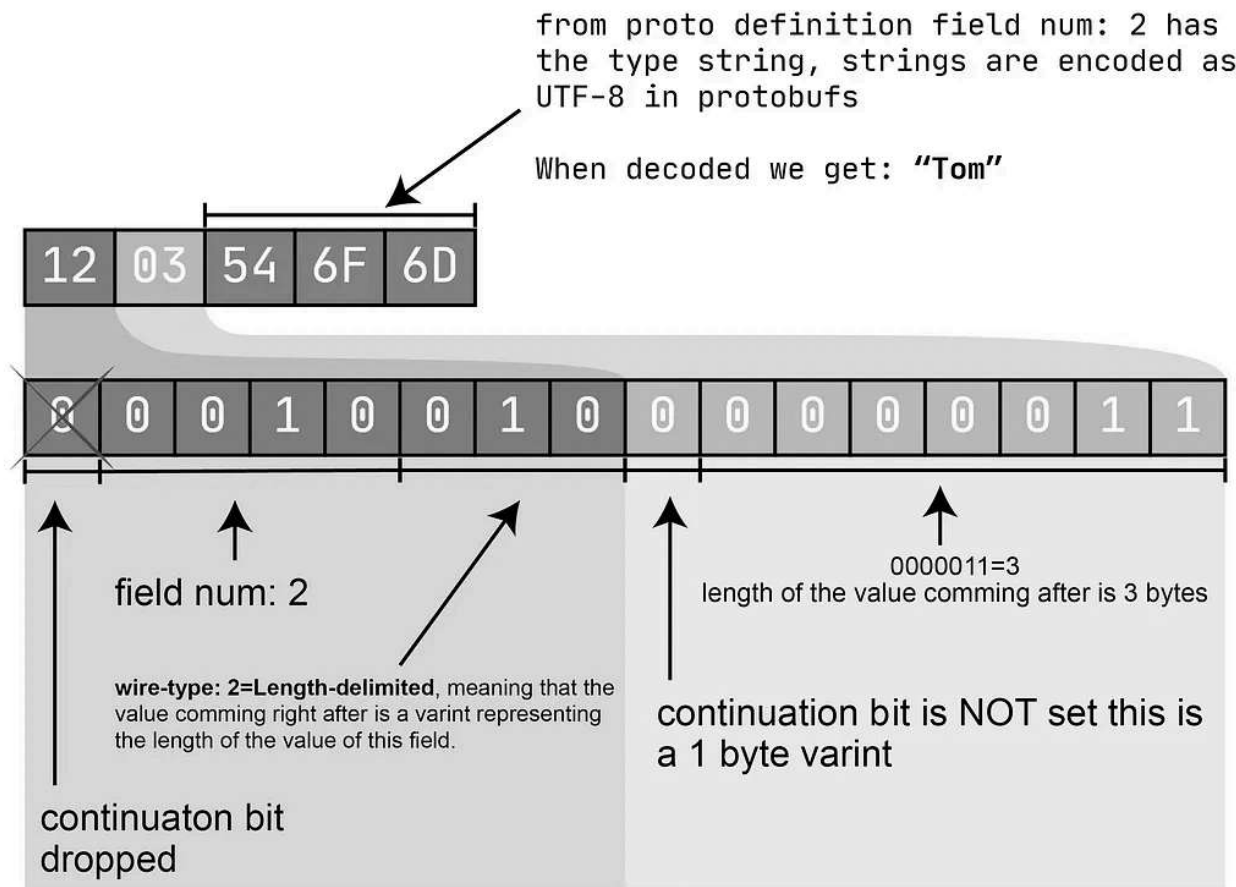
- Get field names based on field numbers
- Understand how to decode wire-types and load them in proper language native types.



There is a lot more to discuss about Protocol Buffers but, for now let's finish with examining the byte stream, to also understand how Length-delimited fields are encoded.

We are at fourth byte `12` which is `00010010` so we've got wire-type `010 = 2` Length-delimited, field number `0010 = 2`. The type here tells us that we should expect a varint that represents length of the value, the next byte `03` which is `00000011` in binary, it's a var int that equals 3, we can go ahead and read next 3 bytes of the value

54 6F 6D . From proto we get field name "name", field type is string , strings are encoded as UTF8, when decode the value with UTF8 we get "Tom".



field num: 2 = "name"

Main thing to note about Length-delimited fields is that it has the length varint sitting between the key and the value.

Have a nice protobuffing! 🤖

Level Up Coding

Thanks for being a part of our community! Before you go:

- 👏 Clap for the story and follow the author 🙌
- 📖 View more content in the [Level Up Coding](https://levelup.gitconnected.com/protobufs-explained-17ed6ba52076) publication

-  Follow us: [Twitter](#) | [LinkedIn](#) | [Newsletter](#)

  [Join the Level Up talent collective and find an amazing job](#)



Follow

Published in Level Up Coding

172K Followers · Last published 5 hours ago

Coding tutorials and news. The developer homepage [gitconnected.com](#) && [skilled.dev](#) && [levelup.dev](#)



Follow

Written by Giga Gatenashvili

11 Followers · 10 Following

No responses yet



What are your thoughts?

Respond

More from Giga Gatenashvili and Level Up Coding

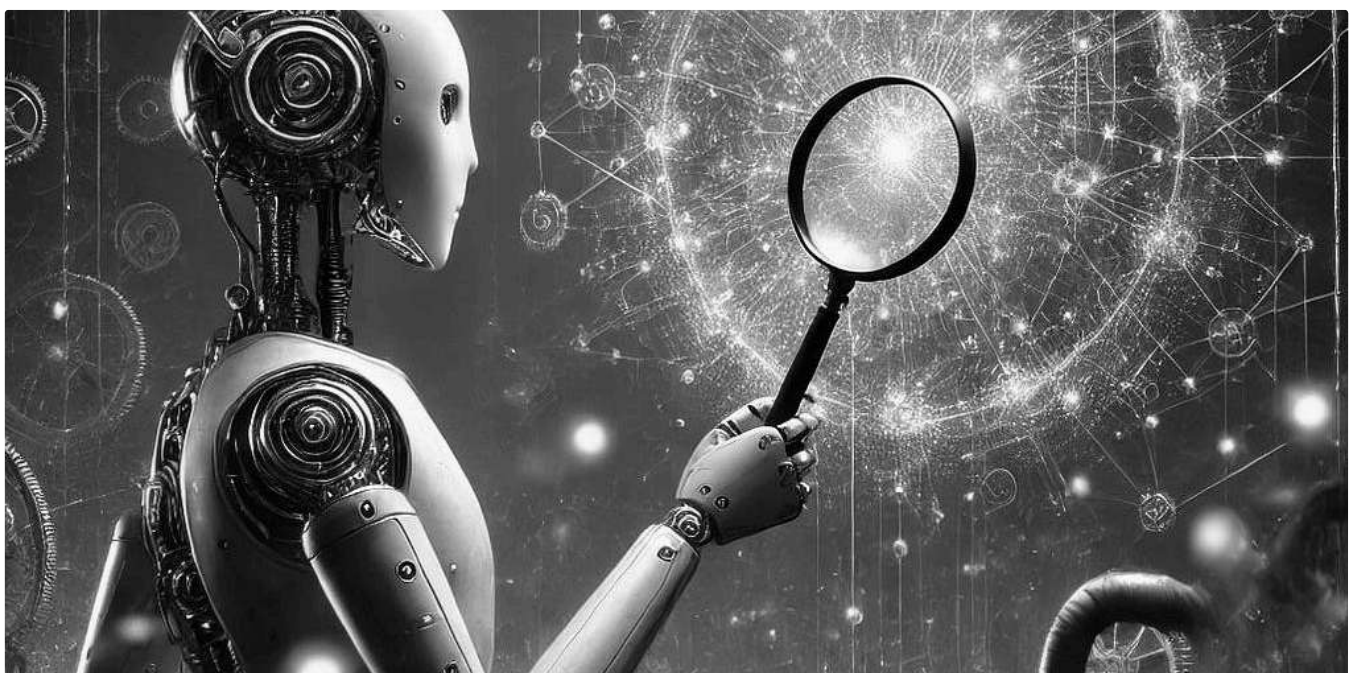


 In Level Up Coding by Giga Gatenashvili

How to Ensure that a Function Accepts a Value but not a Promise in TypeScript

Today I had to write a function in TypeScript that would accept single argument with type object. Let's call this function foo. Then I had...

Oct 20, 2020  86  1

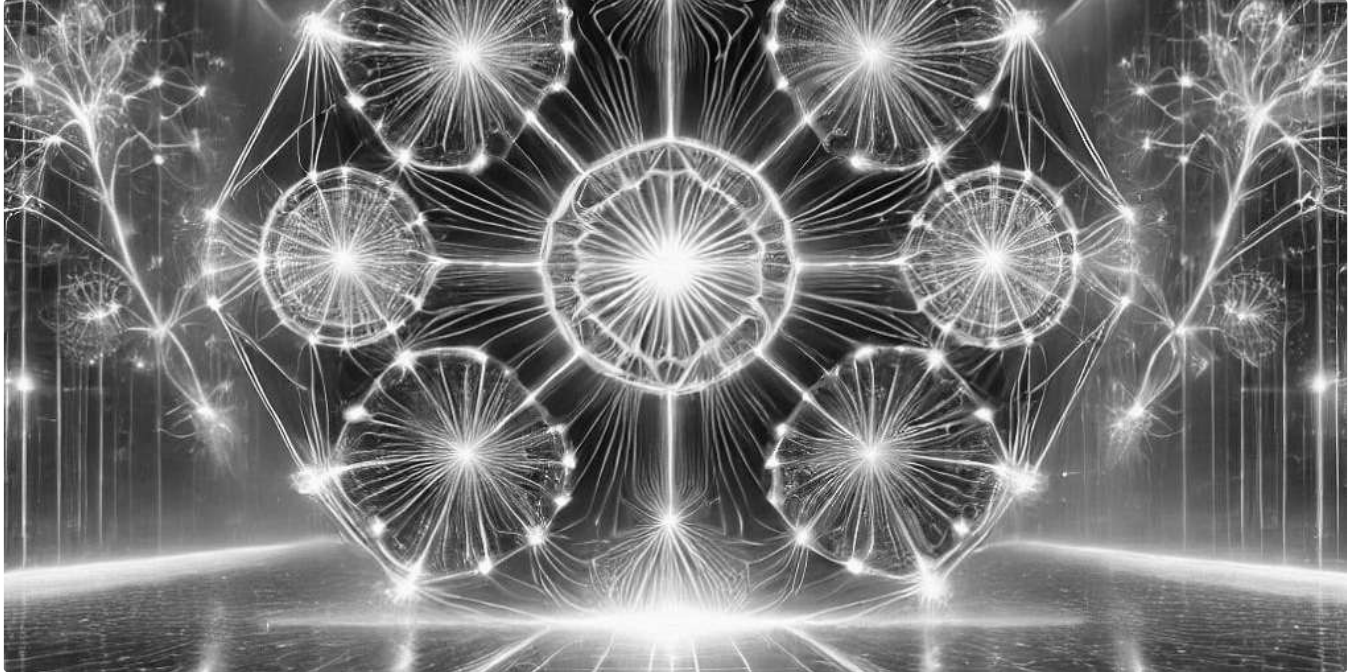


 In Level Up Coding by Salvatore Raieli

Tabula Rasa: A Deep Dive on Kolmogorov-Arnold Networks (KANs)

A Deep Dive into Next-Gen Neural Networks

★ 1d ago 🖱 598 💬 18



 In Level Up Coding by Thomas Reid 

Yet another new experimental Gemini model tops the Chatbot Arena

Is this one any better than its 1 week-old predecessor?

★ Nov 24 🖱 276 💬 3





In Level Up Coding by Bret Cameron

Automate Your Release Notes with AI

How to save time every week using GitLab, OpenAI, and Node.js



Nov 17



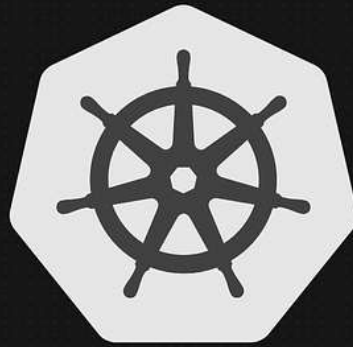
154



See all from Giga Gatenashvili

See all from Level Up Coding

Recommended from Medium



kubernetes



In Stackademic by Crafting-Code

I Stopped Using Kubernetes. Our DevOps Team Is Happier Than Ever

Why Letting Go of Kubernetes Worked for Us



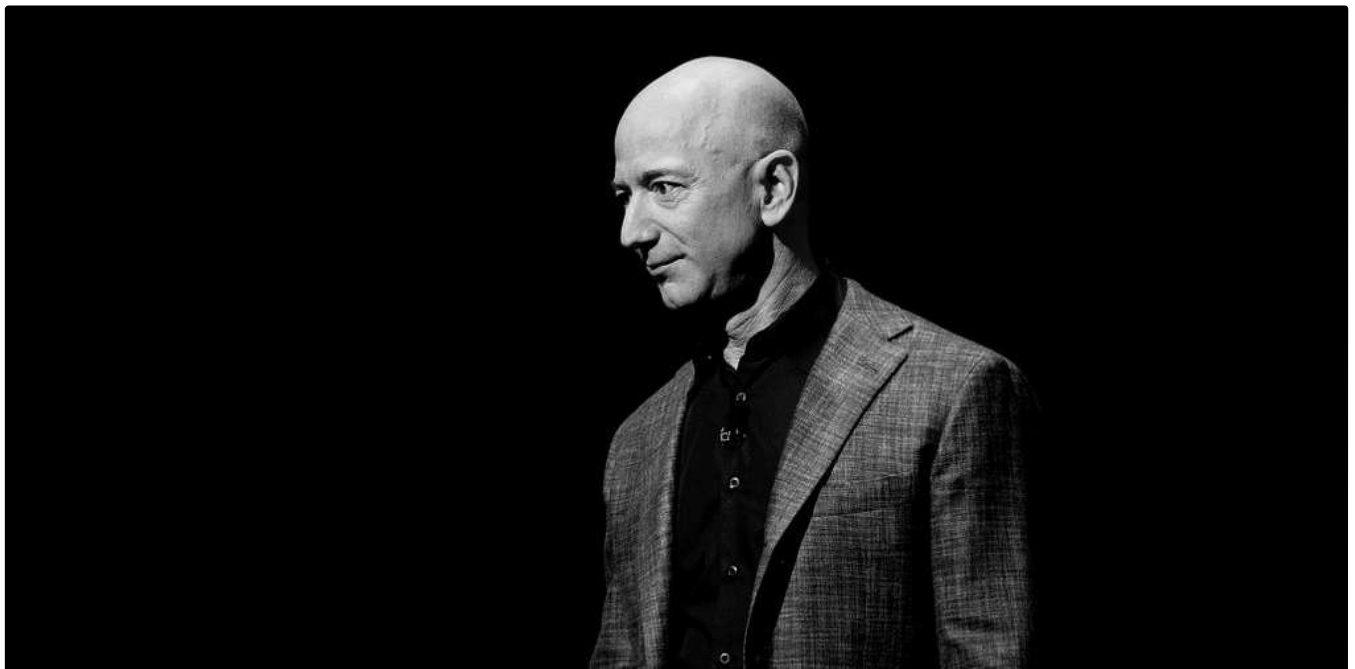
Nov 19



3.7K



111



Jessica Stillman

Jeff Bezos Says the 1-Hour Rule Makes Him Smarter. New Neuroscience Says He's Right

Jeff Bezos's morning routine has long included the one-hour rule. New neuroscience says yours probably should too.

★ Oct 30 🖱 13.9K 💬 324



Lists



Staff picks

778 stories · 1487 saves



Stories to Help You Level-Up at Work

19 stories · 887 saves



Self-Improvement 101

20 stories · 3113 saves



Productivity 101

20 stories · 2617 saves

Always Free

24 GB RAM + 4 CPU + 200 GB



@harendraverma2



@harendra21



@harendra21



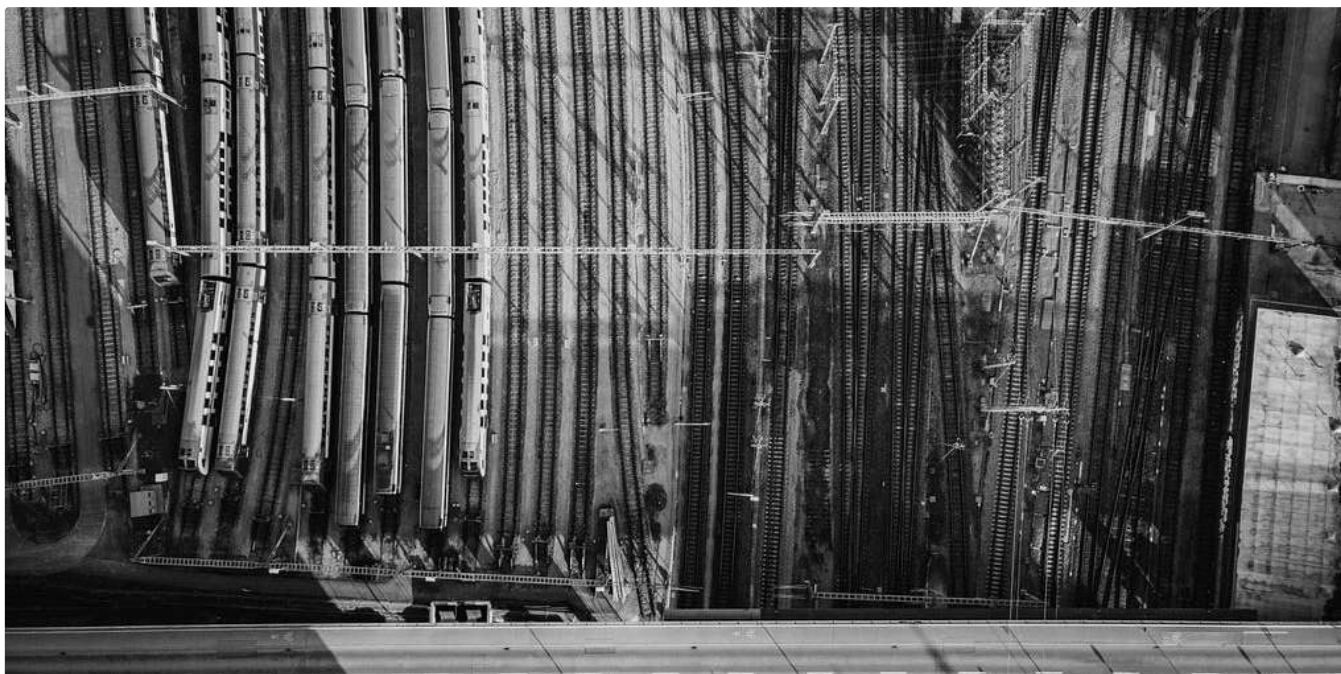
Harendra

How I Am Using a Lifetime 100% Free Server

Get a server with 24 GB RAM + 4 CPU + 200 GB Storage + Always Free

★ Oct 26 🖱 6.2K 💬 90





 Andrew Zuo

Async Await Is The Worst Thing To Happen To Programming

I recently saw this meme about async and await.

✦ Jun 21 🖱 4K 💬 221



 Mark Shrike, MD, PhD

The dumbest decision I ever made (and the Nobel Prize that explains it)

Decision science, Family Guy, and why the “safe” choice is often the riskiest.

✦ Nov 21 🖱 7.7K 💬 176





In ILLUMINATION by Kallol Mazumdar

I Went on the Dark Web and Instantly Regretted It

Accessing the forbidden parts of the World Wide Web, only to realize the depravity of humanity

Mar 13 25K 532



See more recommendations